

Sistema De Gestion De Estudiantes

Juan Andrés Medina Pardo

Estudiante

William Alexander Matallana Porras

Docente

Universidad de Cundinamarca, Extensión Chía

20/02/2026

INTRODUCCIÓN

El desarrollo de software requiere una metodología estructurada que garantice organización, calidad y cumplimiento de objetivos. El Ciclo de Vida del Desarrollo de Software (SDLC) proporciona un marco de trabajo que permite planificar, analizar, diseñar e implementar sistemas de manera ordenada.

En el presente documento se describe el desarrollo de un Sistema de Gestión de Estudiantes, aplicado hasta la fase de codificación del SDLC. El sistema fue desarrollado en Java utilizando el patrón arquitectónico MVC (Modelo – Vista – Controlador), lo que permitió estructurar el código de manera modular, organizada y escalable.

El objetivo principal del sistema es permitir el registro, consulta, modificación y eliminación de estudiantes a través de una aplicación en consola, aplicando buenas prácticas de programación y principios de diseño estructurado.

JUSTIFICACIÓN

La gestión adecuada de información académica es fundamental en cualquier institución educativa. Un sistema organizado permite mejorar la eficiencia administrativa, reducir errores y optimizar el acceso a la información.

La implementación de este proyecto permite:

- Aplicar conocimientos de programación orientada a objetos.
- Comprender el uso práctico del modelo MVC.
- Aplicar las etapas del SDLC en un proyecto real.
- Desarrollar habilidades en estructuración y documentación de software.

Además, este proyecto fortalece la comprensión del proceso completo de desarrollo, desde la planificación hasta la implementación.

DESARROLLO DEL SDLC

1. Planificación (Planning)

En esta etapa se definió el objetivo del sistema: desarrollar una aplicación que permita gestionar estudiantes mediante operaciones básicas como registrar, listar, buscar, modificar y eliminar.

Se determinó el alcance del proyecto:

- Aplicación en consola.
- Almacenamiento en memoria (sin base de datos externa).
- Uso del lenguaje Java.
- Aplicación del patrón MVC.

Resultado: Documento de alcance y objetivos del sistema.

2. Análisis de Requerimientos (Requirements Analysis)

Se definieron los requerimientos funcionales:

- Registrar estudiante.
- Listar estudiantes.
- Buscar estudiante por ID.
- Modificar datos del estudiante.

- Eliminar estudiante.
- Generación automática de ID.

Requerimientos no funcionales:

- Código organizado y modular.
- Interfaz clara y comprensible.
- Cumplimiento de buenas prácticas de programación.

Resultado: Especificación detallada de funcionalidades.

3. Diseño del Sistema (System Design)

Se definió la arquitectura MVC:

Modelo (Model): Representa los datos del sistema.

Vista (View): Maneja la interacción con el usuario.

Controlador (Controller): Conecta la vista con la lógica del negocio.

Servicio (Service): Contiene la lógica de negocio.

Main: Punto de entrada del sistema.

Este diseño permite separación de responsabilidades y facilita el mantenimiento del sistema.

4. Diseño de Datos (Data Design)

Se estructuró la entidad Estudiante con los siguientes atributos:

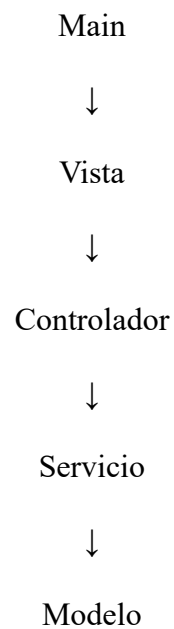
- ID (int)
- Nombre (String)
- Correo (String)
- Programa Académico (Enum)

Se utilizó una lista dinámica (List<Estudiante>) para almacenar los datos en memoria.

El uso de Enum garantiza que los programas académicos sean opciones predefinidas, evitando errores de digitación.

5. Diseño de Arquitectura (Architecture Design)

Flujo del sistema:



Este flujo garantiza que cada capa tenga una responsabilidad específica, evitando acoplamiento innecesario.

6. Diseño de Interfaz (UI Design)

Se diseñó una interfaz en consola con el siguiente menú:

1. Registrar estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Modificar estudiante
5. Eliminar estudiante
6. Salir

La interfaz incluye validaciones básicas y mensajes de confirmación para mejorar la experiencia del usuario.

7. Codificación (Implementation / Codification)

Se implementaron las siguientes clases:

- Estudiante: Representa la entidad principal.
- ProgramaAcademico: Enum con programas académicos.
- EstudianteService: Maneja la lógica de negocio.

- EstudianteController: Intermediario entre vista y servicio.
- EstudianteView: Maneja el menú e interacción.
- Main: Ejecuta la aplicación.

Durante la codificación se aplicaron principios de:

- Encapsulamiento.
- Modularidad.
- Separación de responsabilidades.
- Organización por paquetes.
- Uso de listas dinámicas.

El sistema final permite gestionar estudiantes correctamente desde la consola.

DIAGRAMA EXPLICATIVO DEL MVC (Representación Textual)

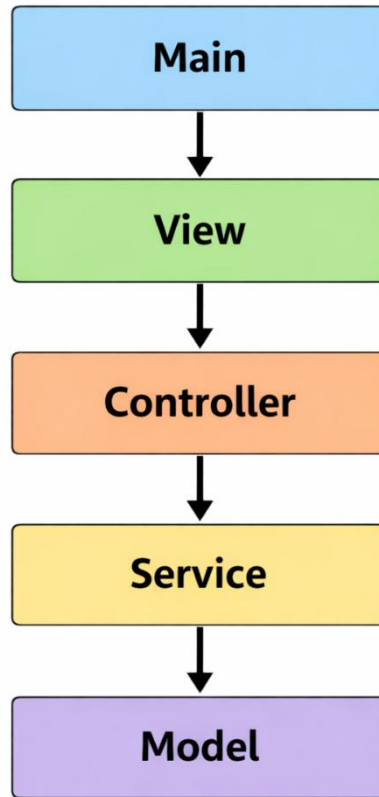


Diagrama de Casos de Uso

En el diagrama de casos de uso se identifica como participante principal al usuario, quien interactúa directamente con el sistema de gestión de estudiantes. Este actor es responsable de ejecutar las funcionalidades disponibles, como registrar, consultar, buscar y eliminar estudiantes, representando la entidad externa que utiliza los servicios que ofrece el sistema.

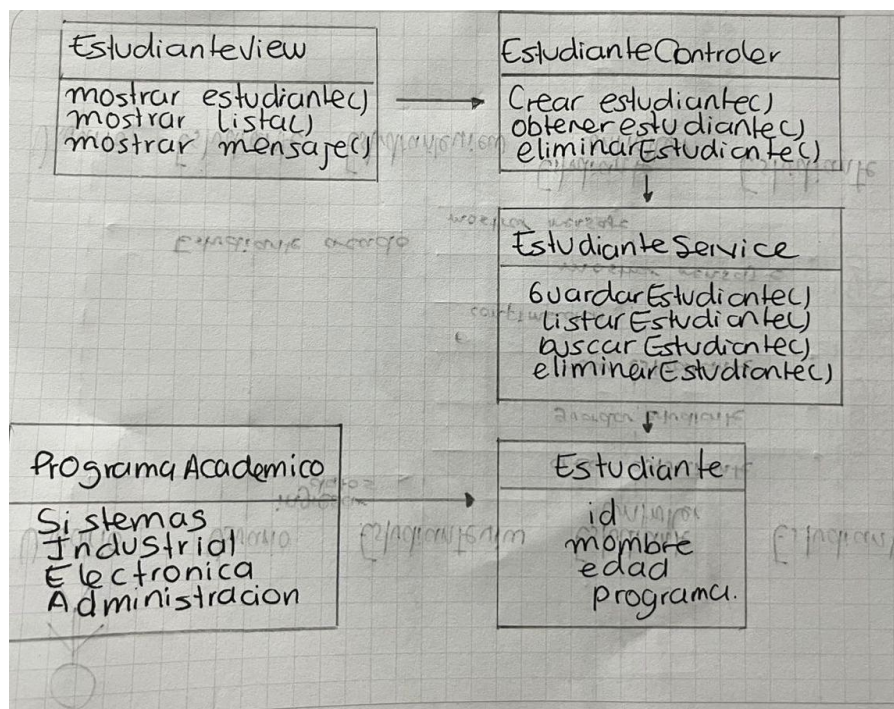


Diagrama De Secuencia

En el diagrama de secuencia se identifican como participantes el usuario, la vista (EstudianteView), el controlador (EstudianteController), el servicio (EstudianteService) y el modelo (Estudiante). Estos elementos interactúan de manera ordenada para llevar a cabo el proceso de creación de un estudiante, evidenciando la comunicación entre la interfaz, la lógica de control, la capa de negocio y el modelo de datos dentro del sistema.

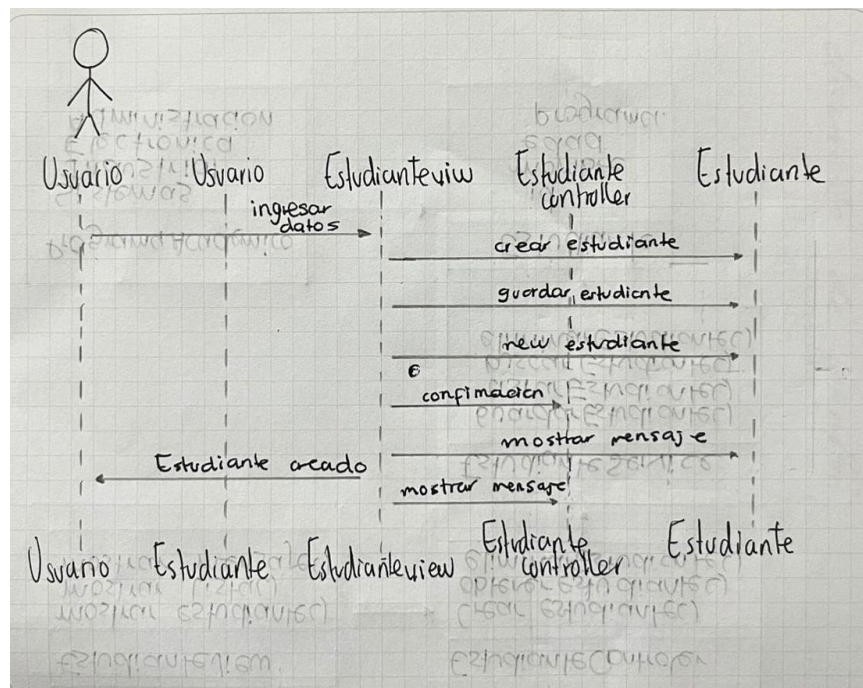


Diagrama de Estados

El diagrama de estados representa el ciclo de vida de un estudiante dentro del sistema de gestión, mostrando los diferentes estados por los que puede pasar su registro y las transiciones que ocurren entre ellos.

El proceso inicia en el estado Nuevo, cuando se ingresa la información del estudiante en el sistema. Posteriormente, al completar su registro, el estudiante pasa al estado Registrado, donde su información queda almacenada y disponible para consulta.

Desde este estado, el registro puede ser Modificado si se realizan actualizaciones en sus datos, o puede pasar a Eliminado cuando se decide removerlo del sistema. Estas transiciones reflejan las operaciones principales que se ejecutan durante la administración de estudiantes.

Este diagrama permite comprender cómo evoluciona el estado de un estudiante a lo largo del tiempo, facilitando la visualización del comportamiento dinámico del sistema y el control del ciclo de vida de la información gestionada.

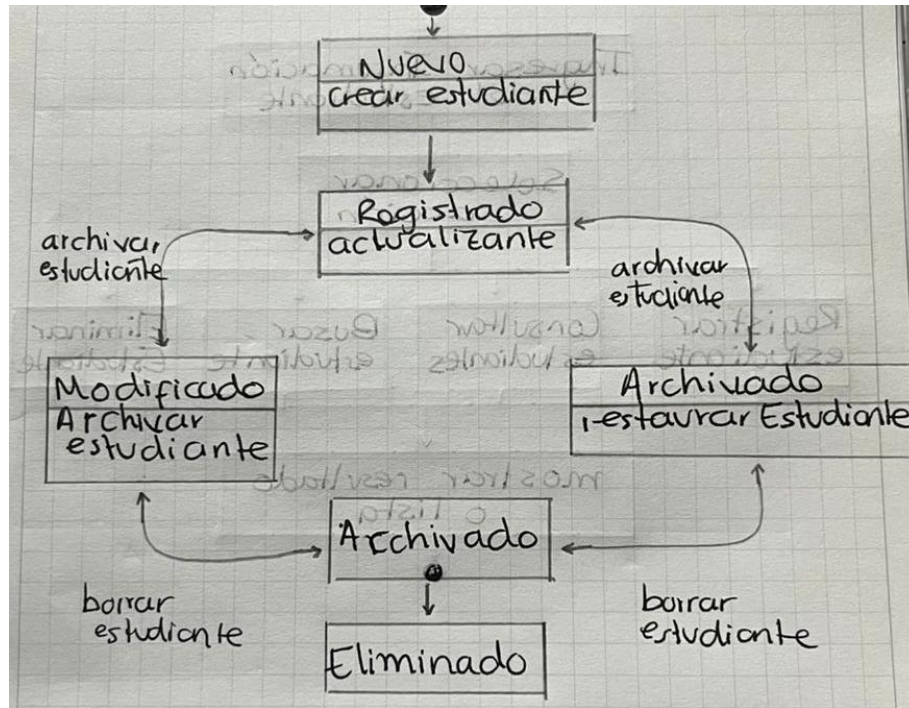
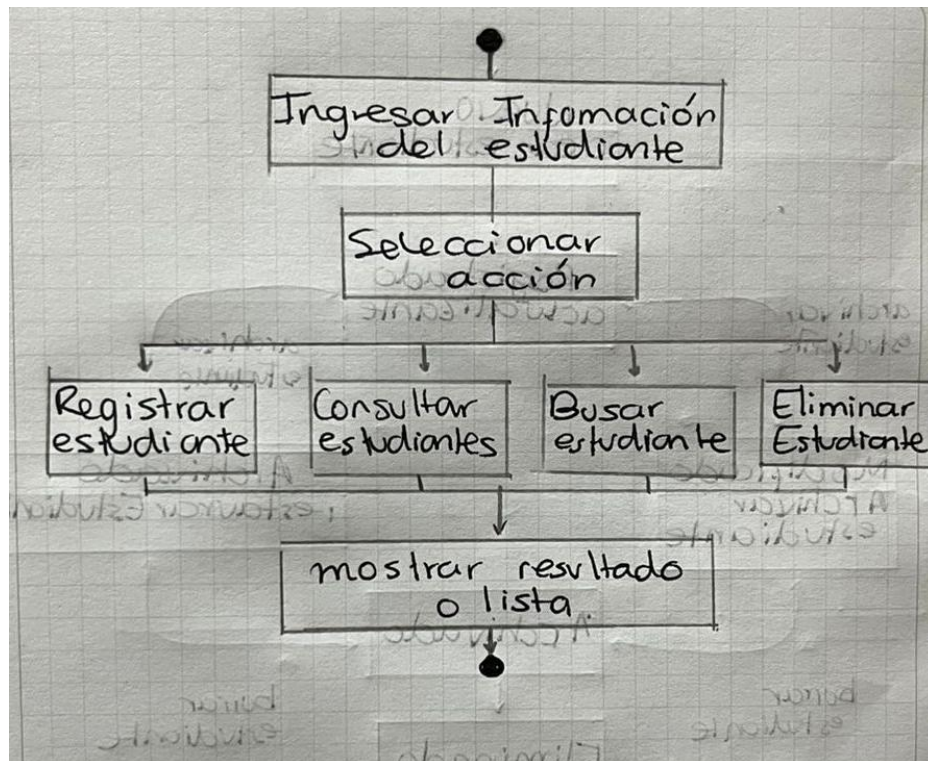


Diagrama De Actividad

El diagrama de actividades representa el flujo de trabajo del sistema de gestión de estudiantes al ejecutar una funcionalidad, mostrando las acciones, decisiones y el orden en que se realizan las tareas dentro del proceso.

En este caso, el diagrama describe el proceso de creación de un estudiante. El flujo inicia cuando el usuario ingresa la información del estudiante en la interfaz del sistema. Posteriormente, el sistema valida que los datos sean correctos y completos. Si la validación es exitosa, se procede a registrar el estudiante y almacenarlo en el sistema. Finalmente, se muestra un mensaje de confirmación indicando que el estudiante fue creado correctamente, concluyendo así el proceso.

Este diagrama permite comprender de forma clara la secuencia de actividades, los puntos de decisión y el recorrido que sigue la información desde que el usuario inicia la acción hasta que el sistema genera el resultado.



Formato De Requerimientos Del Sistema

Campo	Información
Nombre del proyecto	Sistema de Gestión de Estudiantes
Integrantes	Juan Andrés Medina Pardo
Programa académico	Ingeniería de Sistemas y Computación
Fecha	17/02/2026

Lenguaje de programación	Java
Tipo de aplicación	Consola

2. Descripción general del sistema

- El sistema permite registrar estudiantes con información básica académica.
- Permite consultar, modificar y eliminar estudiantes registrados.
- Genera identificadores automáticos para cada estudiante.
- La aplicación está desarrollada en Java aplicando Programación Orientada a Objetos y el patrón MVC.

3. Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Entrada(s)	Proceso	Salida
RF-01	Registrar estudiante	Permite registrar un nuevo estudiante	nombre, correo, programa	Genera ID y guarda en lista	Estudiante registrado
RF-02	Listar estudiantes	Muestra todos los estudiantes	—	Recorre lista	Lista de estudiantes
RF-03	Buscar estudiante	Permite buscar por ID	ID	Busca en lista	Estudiante encontrado
RF-04	Modificar estudiante	Permite actualizar datos	ID, nuevos datos	Actualiza objeto	Estudiante modificado
RF-05	Eliminar estudiante	Permite eliminar estudiante	ID	Elimina de lista	Estudiante eliminado

4. Requerimientos No Funcionales (RNF)

ID	Tipo	Descripción
RNF-01	Validación	No se permiten nombres vacíos
RNF-02	Validación	El correo debe tener formato válido
RNF-03	Estructura	Organización por paquetes MVC
RNF-04	Calidad	Aplicación de encapsulamiento y modularidad

5. Relación Requerimiento – POO

ID Requerimiento	Clase	Método	Tipo
RF-01	EstudianteService	registrar()	Funcional
RF-02	EstudianteService	listar()	Funcional
RF-03	EstudianteService	buscarPorId()	Funcional
RF-04	EstudianteService	modificar()	Funcional
RF-05	EstudianteService	eliminar()	Funcional
RNF-01	EstudianteView	validarNombre()	No funcional

CONCLUSIONES

La aplicación del SDLC permitió desarrollar el sistema de manera estructurada y organizada. Cada etapa cumplió una función fundamental dentro del proceso de desarrollo.

El uso del patrón MVC permitió:

- Mejor organización del código.
- Mayor facilidad de mantenimiento.
- Separación clara de responsabilidades.
- Escalabilidad futura del sistema.

El proyecto demuestra la importancia de seguir metodologías formales en el desarrollo de software, incluso en proyectos académicos, ya que garantiza calidad y profesionalismo.

El sistema queda preparado para futuras etapas como pruebas formales, implementación en base de datos y despliegue.

REFERENCIAS BIBLIOGRÁFICAS

Sommerville, I. (2016). Ingeniería de software (10ª ed.). Pearson Educación.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Oracle. (2023). The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/>

Guru99. (s. f.). *Diagrama de actividades UML*. Guru99. <https://www.guru99.com/es/uml-activity-diagram.html>

DiagramasUML. (s. f.). *Diagrama de actividades UML*. DiagramasUML. <https://diagramasuml.com/actividades/>

UETITC. (s. f.). *UML State diagram*. UETITC Software Design I. <https://uetitc.github.io/SoftwareDesignI-2024-2/Lessons/14-UML-State.html>

Aprende Industrial. (s. f.). *Diagrama de clase: simbología*. Aprende Industrial. <https://aprendeindustrial.com/diagrama-de-clase-simbologia/>

Lucidchart. (s. f.). *Tutorial de diagrama de clases UML*. Lucidchart. <https://www.lucidchart.com/pages/es/tutorial-de-diagrama-de-clases-uml>

Modo Debug. (s. f.). *Diagramas de casos de uso*. Modo Debug.

<https://mododebug.com/diagramas-casos-de-uso/>